BusXTrack

Smart College Bus Tracking and ETA System

Project Design, GUI Prototype and MySQL Database Report
Mid-Term I Project-Based Evaluation

Department of Computer Engineering
Academic Year 2026

1 Executive Summary

BusXTrack is a beginner-friendly Java Swing and MySQL application for a college bus office. It helps an administrator maintain buses, drivers, routes, stops and timetables. A student can search for a bus or a stop and see its route, driver details and schedule-based simulated ETA.

The prototype does not use GPS, Google Maps or external APIs. It reads the current time from the computer running the application. It then compares that time with a timetable departure time and the configured minutes of each stop. For example, if a bus departs at 08:00 and Market Road is 15 minutes from the route start, the expected arrival is 08:15. At 08:10, the interface shows approximately 5 minutes remaining.

Item	Project decision
Domain area	College transportation and timetable management
Technology	Java, Swing, JDBC and MySQL
Innovation	Simple schedule-based ETA simulation for a college bus workflow
Expected outcome	A working GUI prototype connected to a normalized MySQL database
Area of utilization	College transport office and students checking bus schedules

2 Problem Statement, Objectives and Scope

Students often do not know which college bus serves a particular stop, when it is expected, or which driver is assigned. Manual timetable sheets can be difficult to search and update. BusXTrack provides one simple desktop interface for maintaining this information and presenting it in a student-friendly way.

Objectives

- Store bus, driver, route, stop and timetable information in MySQL.
- Provide separate admin and student workflows in Java Swing.
- Allow a student to search by bus number or stop name.
- Calculate an estimated arrival time from a fixed schedule and the current system time.
- Demonstrate database integration through JDBC and simple DAO classes.

Scope

- Included: local desktop prototype, CRUD forms, search, route view, driver details, timetable and simulated ETA.
- Excluded: live GPS, maps, mobile deployment, notifications, online authentication and real-time vehicle sensors.
- The prototype is designed for one college and a small sample dataset.

3 Requirements and Feasibility

Functional Requirements

User	Requirement	Prototype screen
Admin	Login with username and password	LoginFrame
Admin	Add, edit and delete buses	BusManagementFrame
Admin	Manage drivers	DriverManagementFrame
Admin	Manage routes and stops	RouteManagementFrame and StopManagementFrame
Admin	Manage departure timetable	TimetableManagementFrame
Student	Search by bus number	SearchBusFrame
Student	Search by stop name	SearchStopFrame
Student	View route and driver details	RouteViewFrame and BusDetailsFrame
Student	View simulated ETA and bus status	Search result tables

Non-functional Requirements

- The interface should be simple enough for a student to use without training.
- Validation should stop empty form submissions and show a message to the user.
- The code should remain readable and easy to explain in a viva.
- The application should work on a local computer with Java, MySQL and the JDBC connector.

Feasibility Study

Type	Finding
Technical	Feasible because Java Swing, JDBC and MySQL are available and suitable for a desktop prototype.
Operational	Feasible for a college transport office because the admin maintains the timetable and students only search information.
Economic	Low cost because the project uses free/open-source development tools and local software.
Schedule	Feasible for a semester mini-project because the scope is limited to fixed schedules and basic CRUD.

4 System Architecture and Module Design

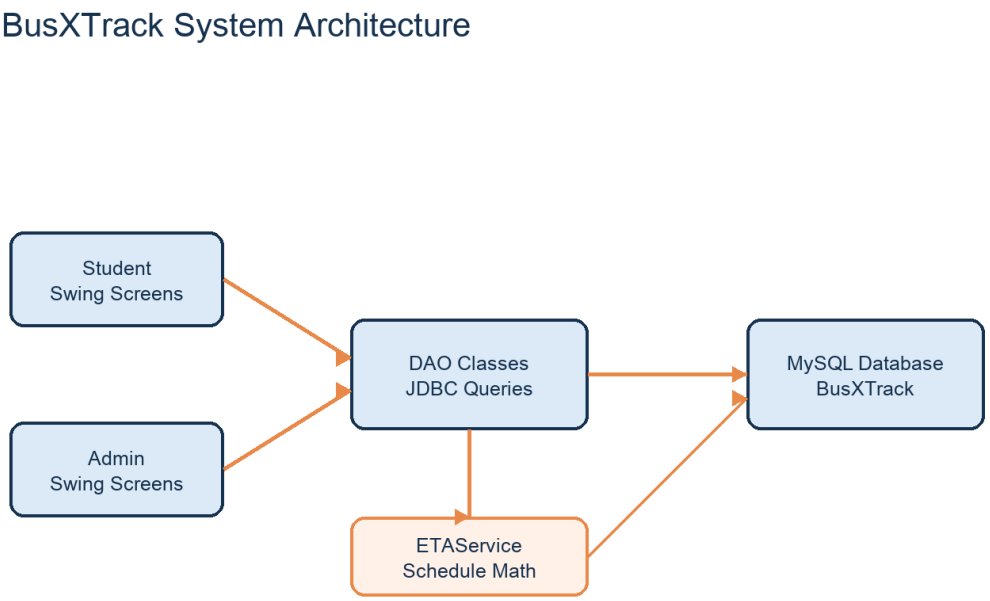


Figure 1. Layered architecture used in the prototype

The user interface contains Swing frames. DAO classes contain the database operations and use DBConnection to create a JDBC connection. Model classes represent records such as Bus and Driver. ETAService contains the schedule calculation so that the formula can be tested and explained separately from the GUI.

Module	Main classes	Responsibility
Models	Admin, Bus, Driver, Route, Stop, Timetable	Store data for one entity using simple fields and getters/setters.
Database	DBConnection	Create a connection to the local busxtrack MySQL database.
DAO	AdminDAO, BusDAO, DriverDAO, RouteDAO, StopDAO, TimetableDAO	Run prepared SQL queries for login, CRUD and search.
Service	ETAService	Compare current time with departure and stop minutes.

GUI	LoginFrame, AdminDashboardFrame, StudentHomeFrame and child frames	Collect input, show tables and navigate between workflows.

5 UML and System Design Diagrams

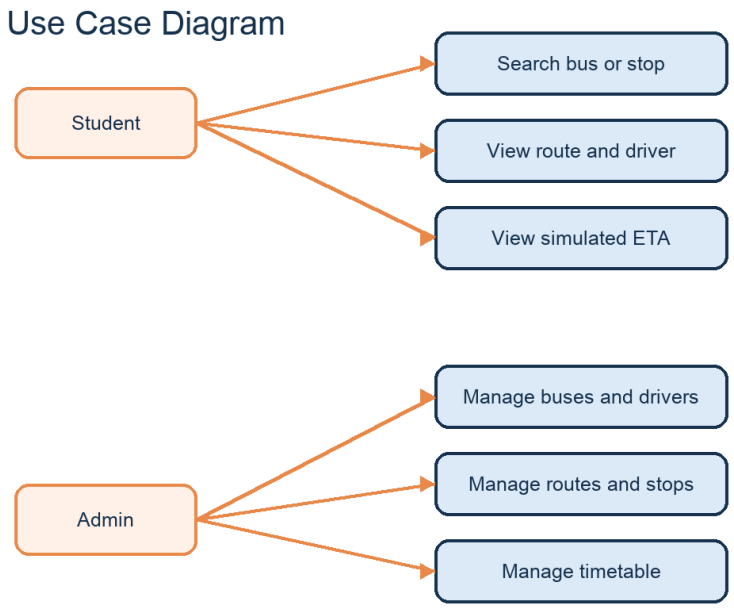


Figure 2. Main user actions

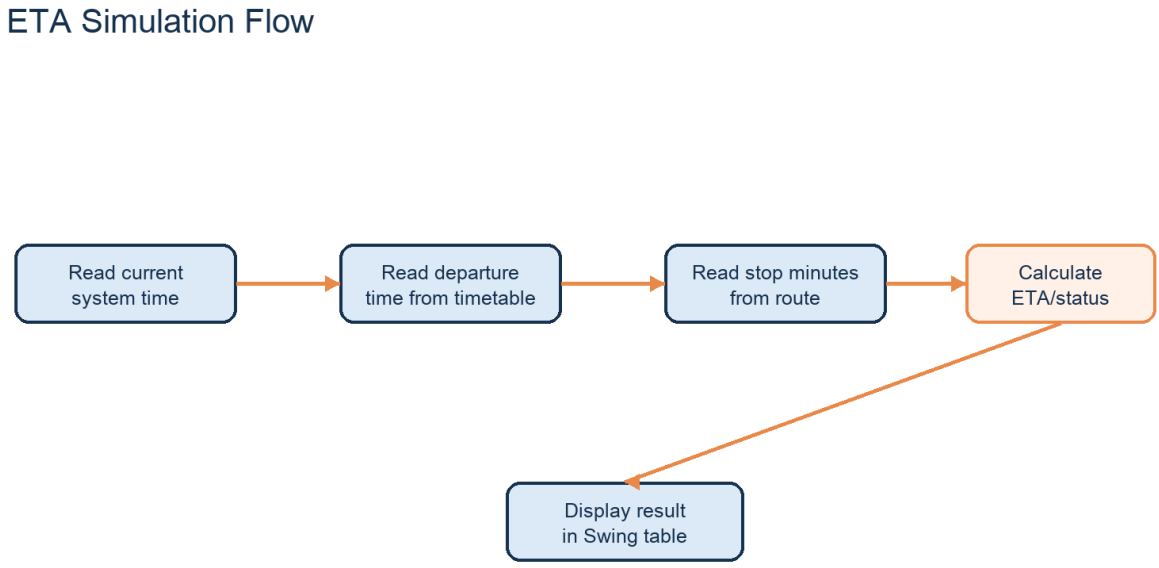


Figure 3. Schedule-based ETA calculation flow

The ETA is calculated as: expected stop time = timetable departure time + minutes_from_start. Remaining time = expected stop time - current system time. The application also compares the current time with the first and last stop to show NOT STARTED, RUNNING or COMPLETED.

6 Java Swing GUI Prototype

Screen	What can be demonstrated
LoginFrame	Admin login with validation; student can continue without admin login.
AdminDashboardFrame	Buttons open bus, driver, route, stop and timetable management.
BusManagementFrame	Form and table for bus number, route, driver and capacity.
DriverManagementFrame	Form and table for driver name, phone and license.
RouteManagementFrame / StopManagementFrame	Create routes and attach ordered stops with minutes from start.
TimetableManagementFrame	Assign a departure time and trip type to a bus.
StudentHomeFrame	Student workflow entry point.
SearchBusFrame	Shows trip, departure, status, next stop and ETA.
SearchStopFrame	Finds buses that include a selected stop and shows expected arrival.
RouteViewFrame / BusDetailsFrame	Shows route stops and assigned driver information.

Navigation and Demonstration Workflow

Application start -> Login screen -> choose Continue as Student or Admin Login. Student -> Search Bus / Search Stop / View Routes -> select or enter data -> view schedule-based result. Admin -> Dashboard -> choose a management module -> fill form -> Add or Update -> refresh table -> return to dashboard.

Validation

- Login checks that username and password are not empty.
- Search screens check that the search field has a value before querying.
- Details screens require a successful search before opening bus details.
- Database exceptions are displayed in a Swing message dialog.
- The database uses NOT NULL, UNIQUE, PRIMARY KEY and FOREIGN KEY constraints.

7 MySQL Database Design

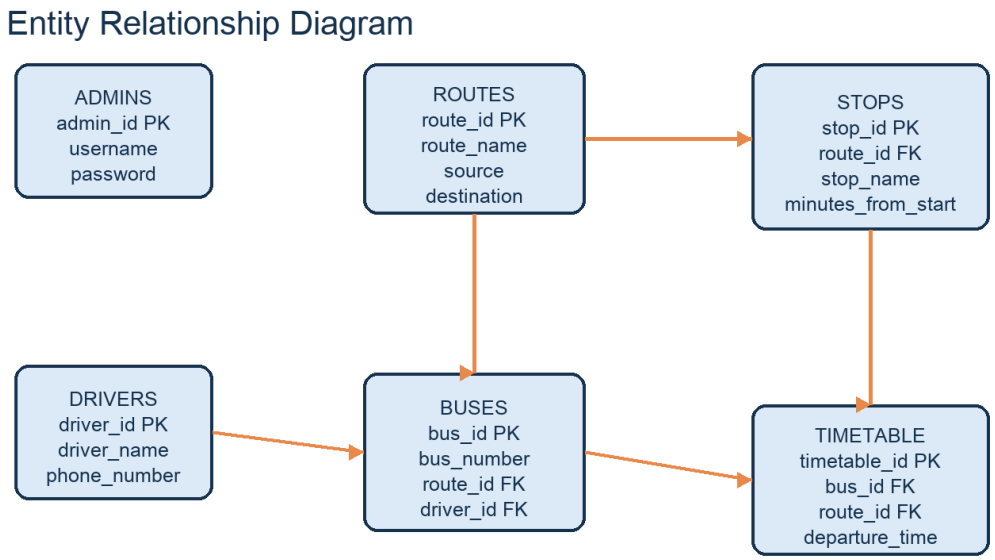


Figure 4. Relational entities and key relationships

Table	Primary key	Important attributes	Foreign keys
admins	admin_id	username, password	None
drivers	driver_id	driver_name, phone_number, license_number	None
routes	route_id	route_name, source, destination	None
stops	stop_id	stop_name, stop_order, minutes_from_start	route_id -> routes
buses	bus_id	bus_number, capacity	route_id -> routes; driver_id -> drivers
timetable	timetable_id	departure_time, trip_type	bus_id -> buses; route_id -> routes

Relationships

- One route has many stops.
- One route can be assigned to many buses.
- One driver can be assigned to a bus record.
- One bus can have multiple timetable records, such as morning and evening trips.
- Foreign keys keep related records consistent and ON DELETE CASCADE removes dependent stops/timetable rows where configured.

Normalization

The schema follows basic normalization. In first normal form, each field stores one atomic value. In second normal form, non-key attributes describe the complete primary key of their table. In third normal form, driver data, route data and stop data are separated instead of being repeated inside the bus or timetable row. Foreign keys connect the entities without storing duplicate descriptions.

Sample SQL Queries

Purpose	Query idea

Find a bus	SELECT * FROM buses WHERE bus_number = ?
List stops	SELECT * FROM stops WHERE route_id = ? ORDER BY stop_order
Find buses for a stop	JOIN stops, buses and routes using route_id
View a bus schedule	SELECT * FROM timetable WHERE bus_id = ?
Show driver details	JOIN buses and drivers using driver_id

The complete executable schema and sample records are in database/busxtrack.sql. The submission folder also includes a readable SQL appendix with demonstration queries.

8 Integration and Current Progress

Component	Current status
Java source structure	Complete prototype source under src/busxtrack with model, DAO, service and UI packages.
Swing navigation	Complete for login, admin dashboard and student search workflow.
MySQL schema	Complete schema with keys, constraints and sample data.
JDBC integration	Implemented through DBConnection and DAO classes; local MySQL credentials may need editing.
ETA logic	Implemented in ETAService using LocalTime and stop minutes.
Documentation	This report, diagrams, SQL appendix, presentation and viva/demo notes.

Limitations and Future Scope

- The current version simulates movement from a timetable and does not know the physical GPS position of a bus.
- The database is local and intended for demonstration on one computer.
- Possible future work includes GPS hardware, a mobile app, notifications, student accounts and an online server.

9 Development Roadmap

Phase	Work	Status
1	Requirements, scope and database design	Completed
2	Create model classes and MySQL schema	Completed
3	Build DAO classes and JDBC connection	Completed
4	Build login, admin and student Swing screens	Completed prototype
5	Connect schedule data to ETAService	Completed prototype
6	Testing, report, diagrams and presentation	Prepared for mid-term
7	Future: improve validation and add live location	Post-mid-term scope

10 Conclusion

BusXTrack demonstrates a complete small desktop application flow. The administrator maintains transport data, the student searches the available information, and the system calculates a transparent ETA from stored schedule values and the current computer time. This scope is realistic for a third-semester Java project because every part can be traced from a Swing input field to a JDBC query and a MySQL table.